

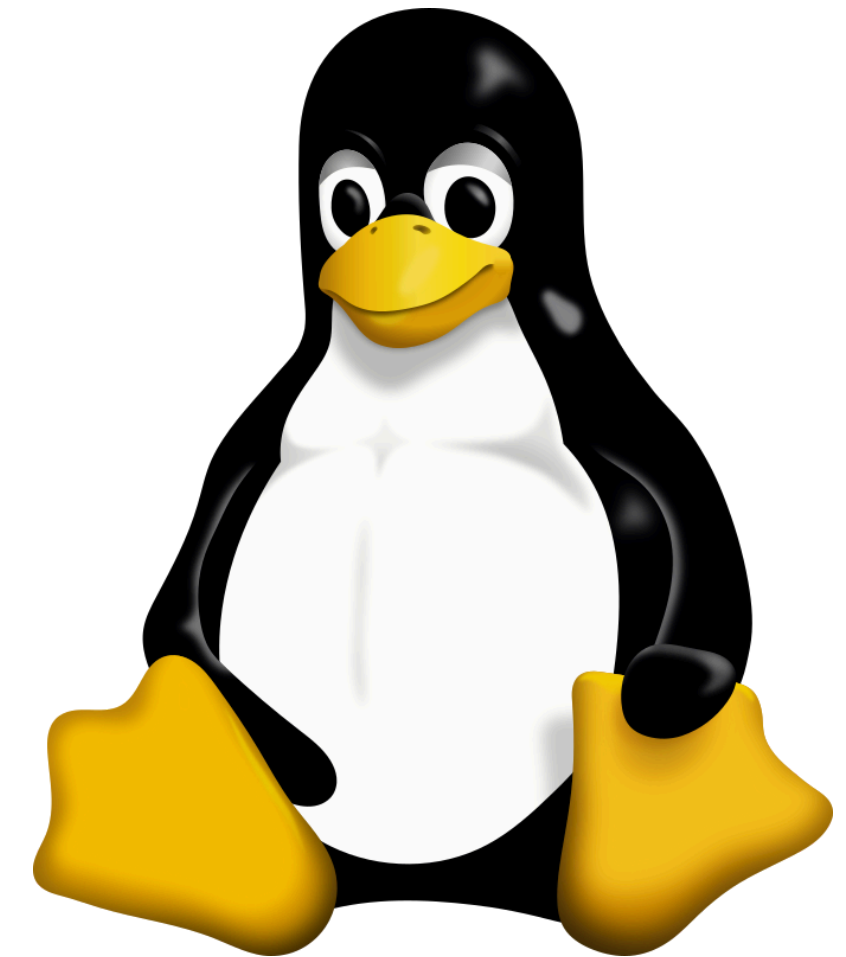
Drawlib

Illustration as Code for Modern Engineers

Declarative Python Diagramming & Modern Presentation Architecture

Yuichi Ito | Drawlib Author

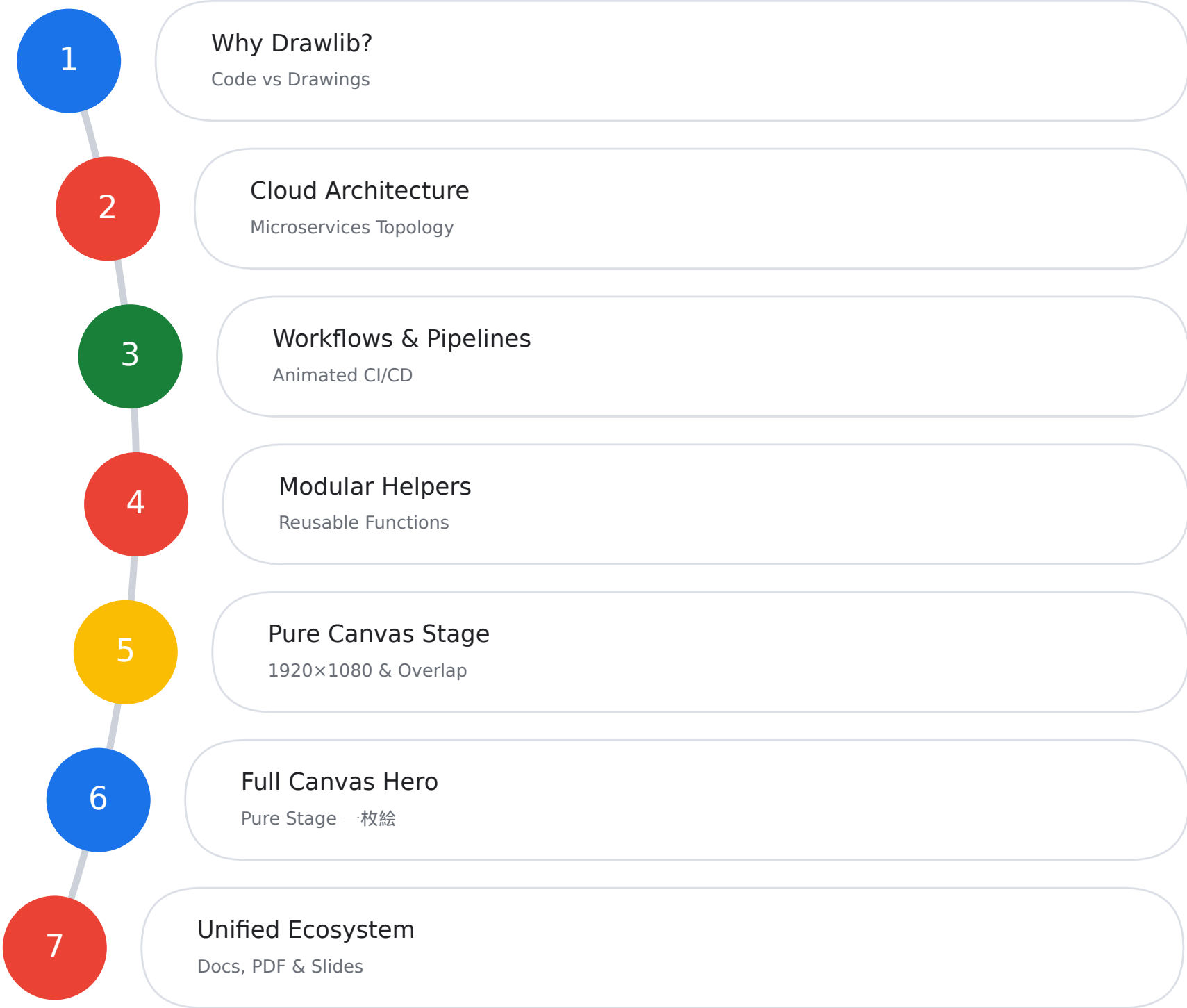
October 2026



Topics Covered

A unified, declarative approach to engineering illustration:

- **Why Drawlib?**: The transition from fragile manual drawings to version-controlled Python code.
- **Cloud Architecture**: Composing microservice topologies with vector icons and orthogonal routing.
- **Workflows & Pipelines**: CI/CD integration, SQLite hash caching, and animated WebP diagrams.
- **Modular SmartArts**: Extensible layout components and custom presentation cards.
- **Pure Canvas Stage**: Absolute 1920×1080 placement, partial bleed overlap, and z-index layers.
- **Full Canvas Hero**: Zero-margin full-bleed illustration stage without boilerplate.
- **Unified Ecosystem**: Documentation sites, vector PDF books, and interactive slide decks.



Why Drawlib?

From Fragile Drawings to Code

Traditional technical diagramming approaches face severe friction:

- **GUI Tools (Visio, Draw.io)** Binary files break Git diffs and code reviews. Diagrams quickly fall out of sync with code.
- **Text DSLs (Mermaid, PlantUML)** Unpredictable layouts, line crossing disasters, and difficult styling overrides.
- **Matplotlib & Graphviz** Heavy low-level boilerplate and unintuitive coordinate systems.

The Drawlib Solution

Pure Python declarative syntax, unified Google semantic palettes, and pixel-perfect coordinate math.

Illustration Approach Comparison

Capability	GUI Tools	Text DSLs	Drawlib
Git & Code Review	Binary diffs (poor)	Text-based (good)	Pure Python (native)
Layout Precision	Manual dragging	Rigid & unpredictable	Pixel-perfect math
Rich Components	Unstandardized	Limited primitives	22 Shapes + Diagrams
CI/CD Automation	Manual export	CLI supported	Native CLI & cache
Design System	Drifts over time	Hard to customize	Global Style Tokens

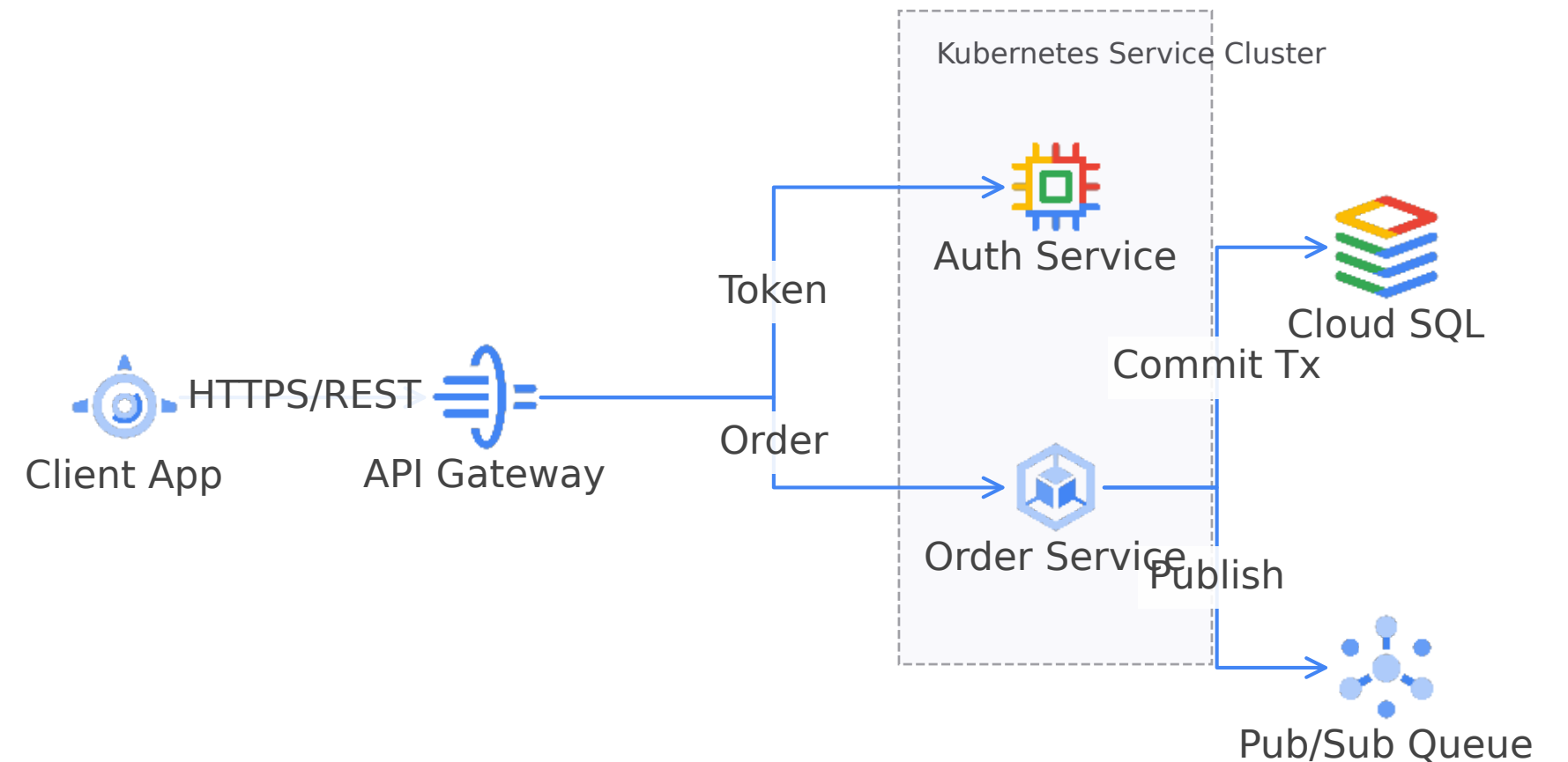
Cloud & Microservices

Scalable Topologies in Code

Drawlib provides high-level domain modules for cloud architecture with vector and official cloud icons:

- **Client Ingress:** Web SPA, Mobile App, Edge Workers terminating TLS
- **Kubernetes Cluster:** Isolated container pods with auto-scaling
- **Data Persistence:** Sharded primary database + Pub/Sub event streaming
- **Clean Syntax:** Connect nodes with labels and automatic orthogonal routing

Scalable Microservices Topology (GCP)

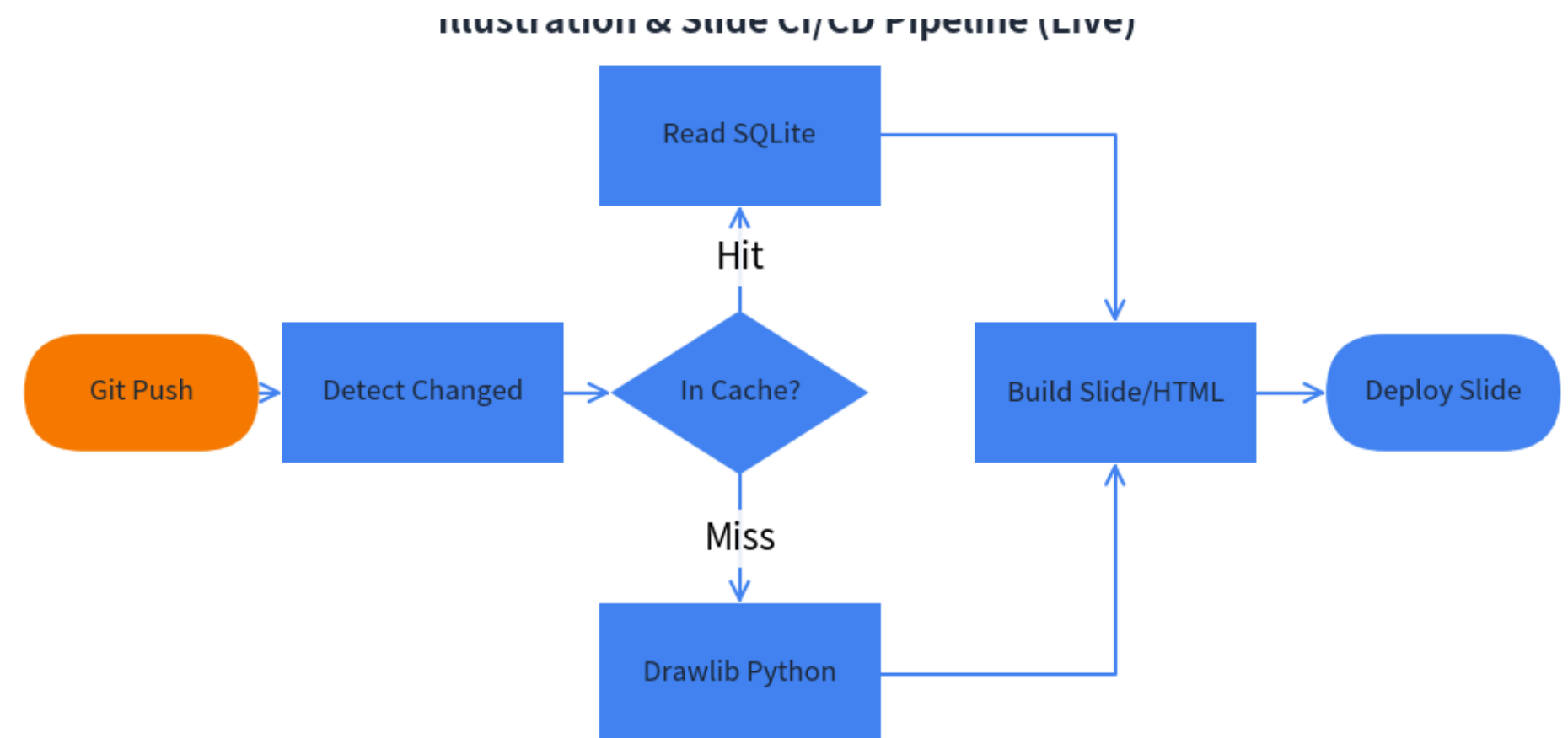


Workflows & Pipelines

Automated CI/CD & Caching

Technical diagrams shouldn't slow down your deployment cycles:

- **Git-Triggered Pipeline:** Build runs automatically on pull requests
- **Change Detection:** Analyzes document diffs and Python dependencies
- **SQLite Hash Caching:**
 - Unchanged blocks are restored in **< 1ms**
 - Only modified diagram code triggers full Python execution
- **Dynamic Animations:** Render interactive APNG / WebP diagrams with click-to-play and frame-level pause (`anim-trigger:click anim-loop:once anim-pause:2,4`)



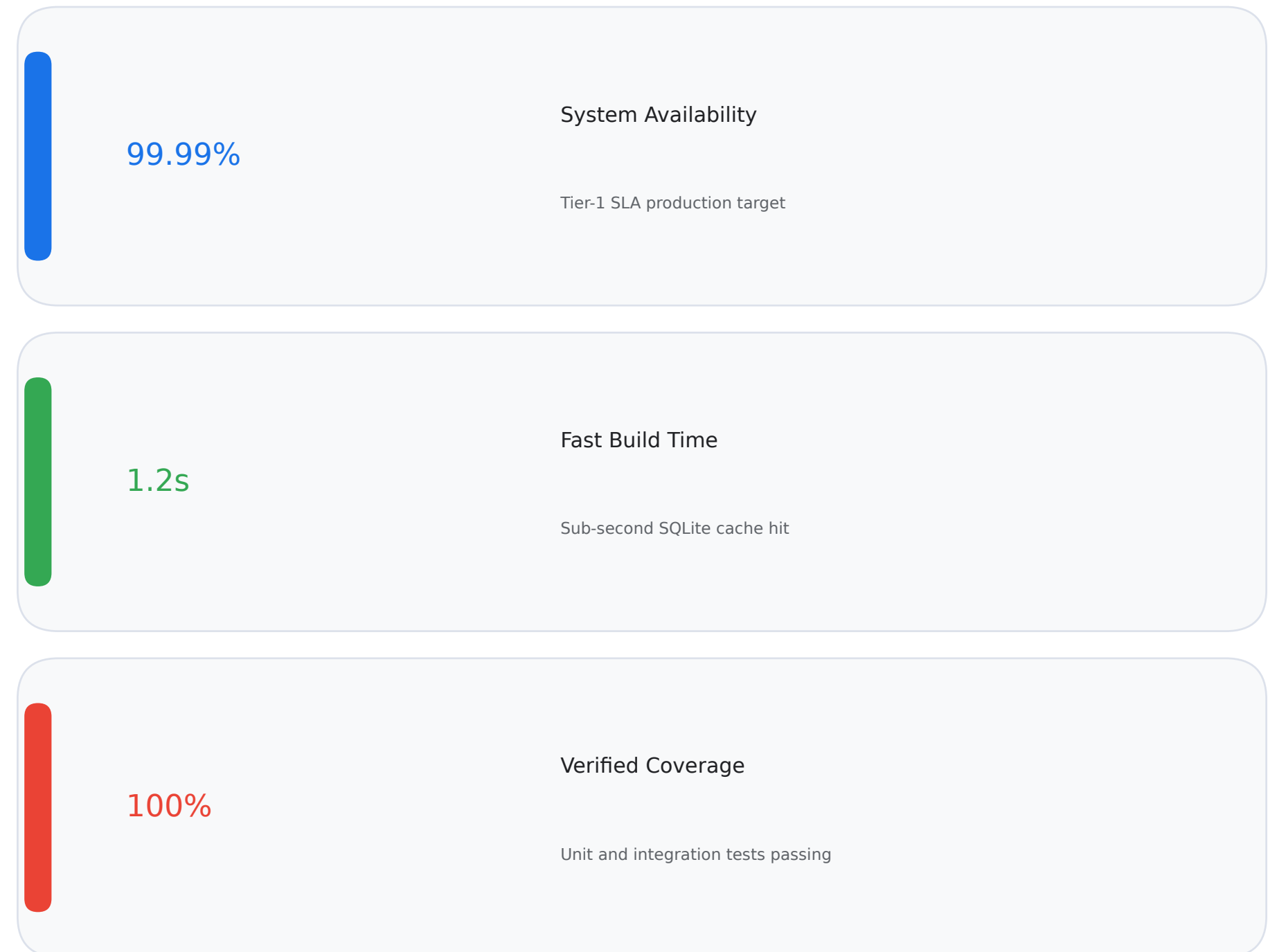
Modular Drawing Helpers

Reusable Python Helpers in utils.py

Drawlib eliminates rigid DSL wrappers in favor of pure, reusable Python functions:

- **Illustration as Code:**
 - Reusable drawing functions reside in project-level `utils.py`
 - Automatically imported and accessible across all documentation & slide decks
 - Full IDE autocomplete, type annotations, and custom styling
- **Built-in Presentation Helpers:**
 - `draw_curved_agenda()` : Curved presentation agenda with numbered badges
 - `draw_kpi_cards()` : Vertically stacked metric cards with accent pills
 - `service_card()` & `connect()` : Microservice architectures and network flows
- **Declarative Invocation:**

Drawlib: Illustration as Code

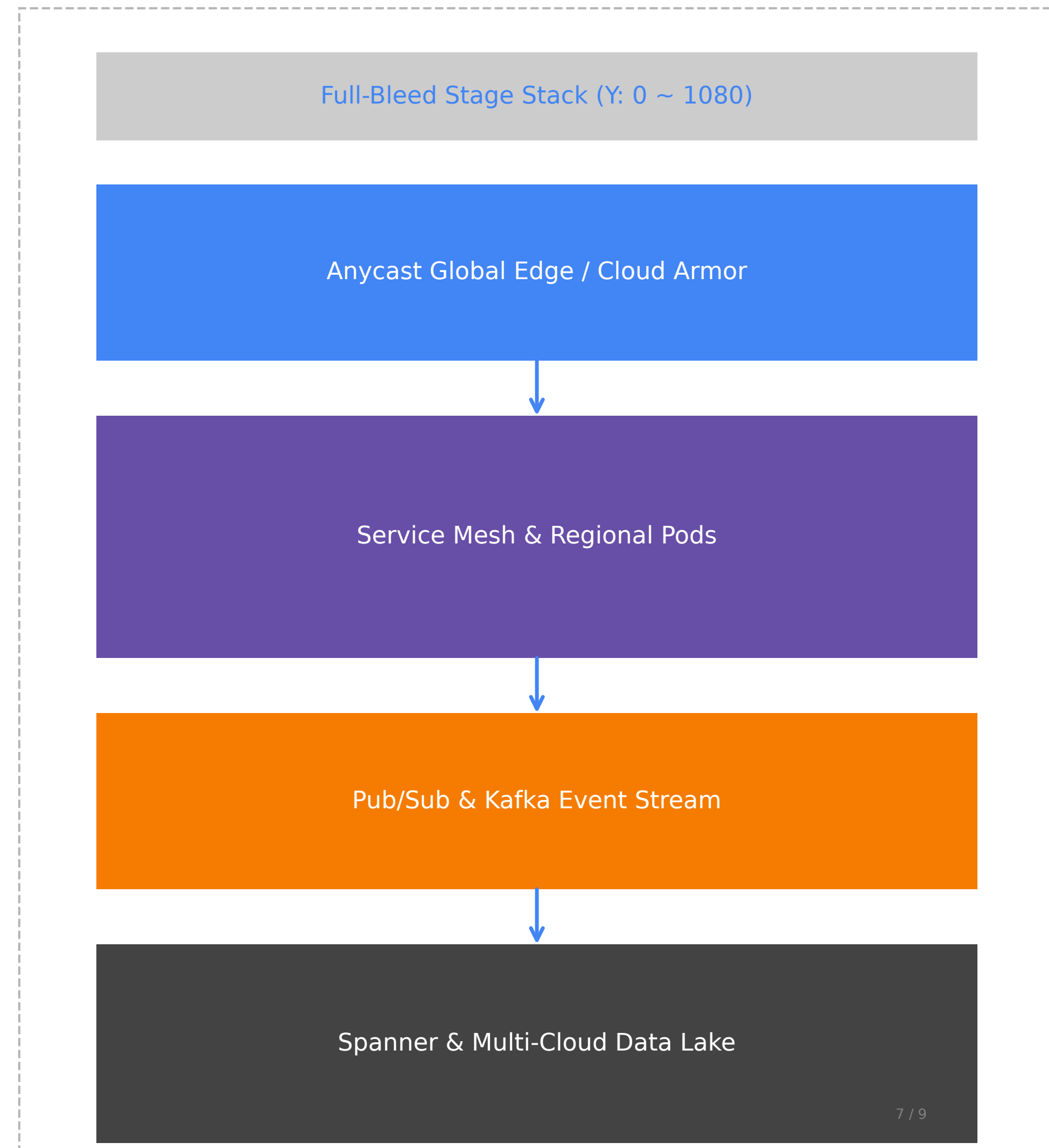


1920×1080 Stage Architecture

Total Spatial Freedom

Every slide is a mathematically defined **1920×1080 Cartesian stage**:

- **Absolute Placement**: Position diagrams, SmartArts, and text boxes at explicit `(x, y)` and `(w, h)` coordinates.
- **Partial Overlap & Bleed**: Notice how the diagram on the right extends beyond the normal content zone, bleeding from `Y: 0` to `Y: 1080` across the stage.
- **Layer Stacking**: Manage depth with `z: N` to compose backgrounds, content panels, and overlays.
- **Modular Scoping**: Use `::: block` for scoped typography and density control (`compact` , `font: 20px`).



FULL CANVAS STAGE — 1920 × 1080 PURE DRAWING STAGE

1. Deterministic

Cartesian geometry
Version-controlled
Reproducible builds

2. Native SVG

Ctrl+F Searchable text
Infinite resolution
Clean DOM integration

3. Multi-Target

Images, PDF Books
Documentation sites
Interactive HTML docs

4. AI Autonomous

Clean token hierarchy
Multi-modal review loop
Structured container syntax

Unified Ecosystem

One Illustration Core, Multiple Targets

Drawlib unifies engineering documentation and technical visualizations as version-controlled code:

Target	Command	Output	Primary Use Case
Standalone Images	<code>drawlib build image</code>	PNG, WebP, PDF	README graphics, blog posts, whitepapers
Documentation Site	<code>drawlib build html</code>	Static Web Site	Technical specs, API references, architecture docs
Document Book	<code>drawlib build pdf</code>	Merged Vector PDF	Formal RFCs, architecture books, print designs
Presentation Deck	<code>drawlib build slide</code>	HTML Slide Deck	Conference talks, design reviews, team pitches

Get Started Today

```
# Initialize a new presentation project with Google styling
uv run drawlib init slide -s google

# Build presentation slides
uv run drawlib build slide slide_src/ -o slide/

# Start interactive preview server
uv run drawlib serve slide/
```